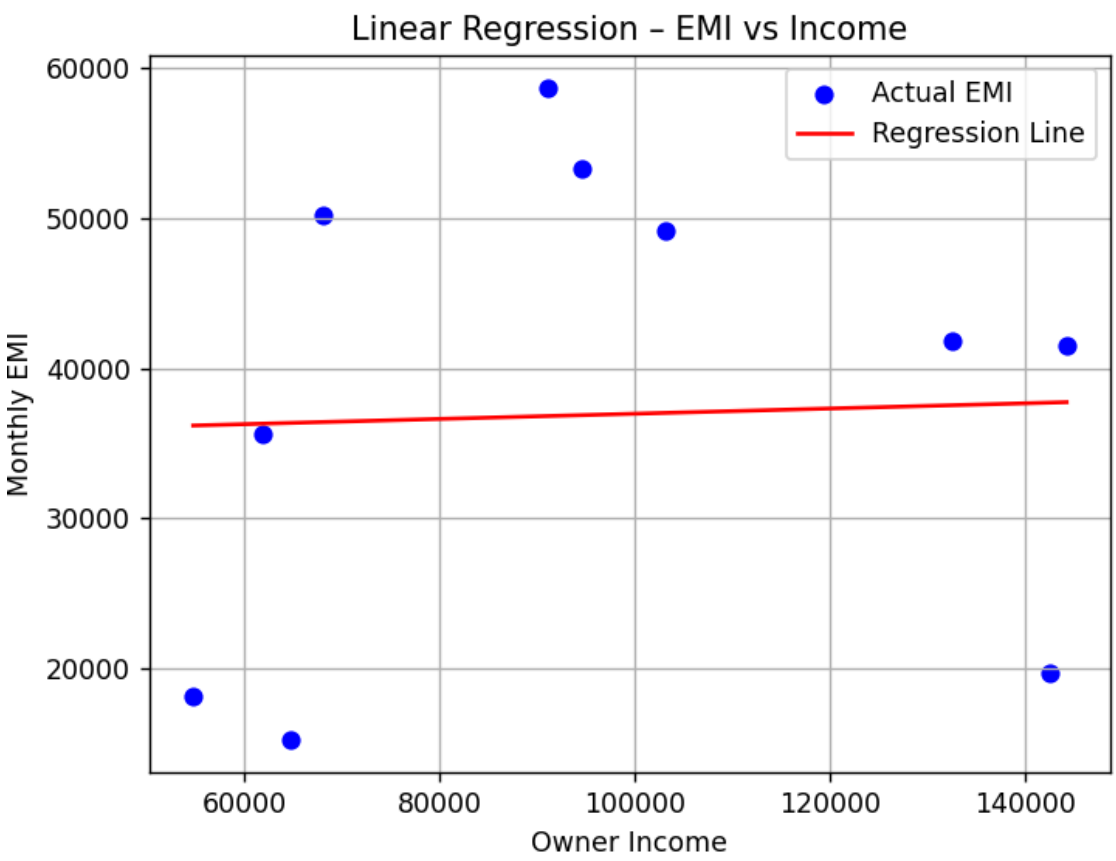
CODING :6

```
import pandas as pd

import matplotlib.pyplot as plt

from sklearn.model_selection import train_test_split

from sklearn.linear_model import LinearRegression

from sklearn.metrics import mean_squared_error, r2_score

data = pd.read_excel(r"C:\Users\ASUS\Downloads\ML PRO 6 dataset.xlsx")

print(data.head())

print(data.columns)

# Feature and target (adjusted to match your dataset)

X = data[['Owner_Income']]      # Predictor

y = data['Monthly_EMI']         # Target

# Split

X_train, X_test, y_train, y_test = train_test_split(

    X, y, test_size=0.2, random_state=42

)

# Model

model = LinearRegression()

model.fit(X_train, y_train)

# Prediction

y_pred = model.predict(X_test)

# Evaluation

print("MSE:", mean_squared_error(y_test, y_pred))

print("R2 Score:", r2_score(y_test, y_pred))

# Sort for clean line

sorted_idx = X_test.values.flatten().argsort()

X_sorted = X_test.values.flatten()[sorted_idx]

y_pred_sorted = y_pred[sorted_idx]

plt.scatter(X_test, y_test, color='blue', label='Actual EMI')

plt.plot(X_sorted, y_pred_sorted, color='red', label='Regression Line')

plt.xlabel("Owner Income")
```

```
plt.ylabel("Monthly EMI")

plt.title("Linear Regression – EMI vs Income")

plt.legend()

plt.grid(True)

plt.show()
```

OUTPUT:

```
   House_ID     City  ...  Monthly_EMI  Owner_Income
0         1    Delhi  ...        52750         48226
1         2   Trichy  ...        31594         84113
2         3  Chennai  ...        46531         66175
3         4   Trichy  ...        24150        111757
4         5  Chennai  ...        51874        136601

[5 rows x 8 columns]
Index(['House_ID', 'City', 'House_Price', 'Loan_Amount', 'Interest_Rate (%)',
       'Loan_Tenure (Years)', 'Monthly_EMI', 'Owner_Income'],
      dtype='object')
MSE: 221999349.75703478
R2 Score: 0.0001204275282647993
```



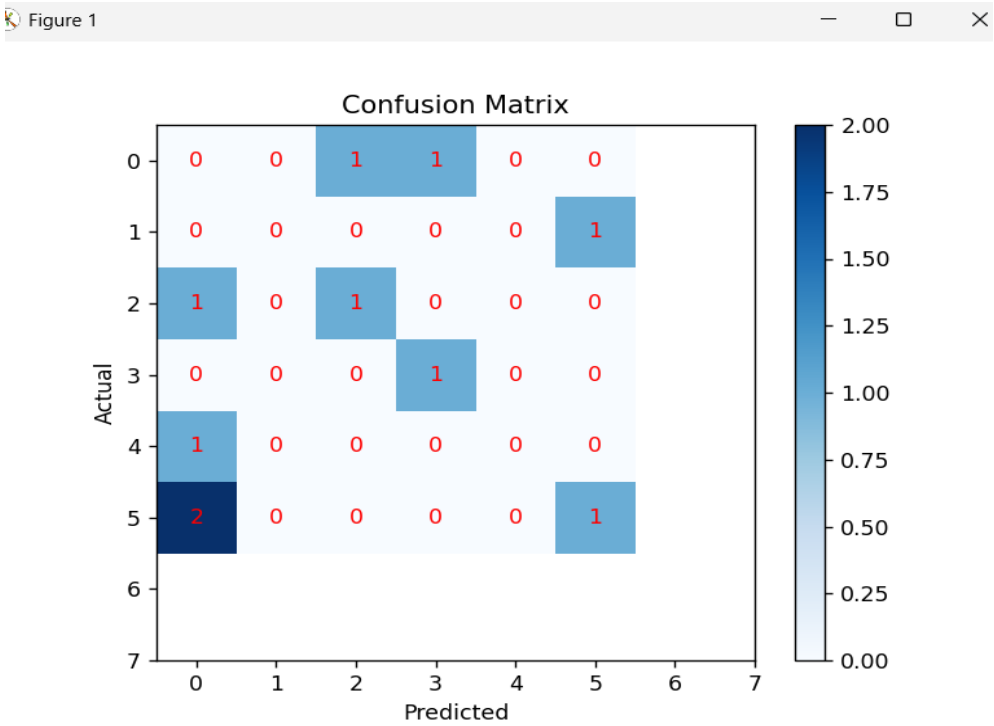
INPUT :7

```
import pandas as pd
import matplotlib.pyplot as plt
import numpy as np
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler, LabelEncoder
from sklearn.svm import SVC
from sklearn.metrics import accuracy_score, confusion_matrix, classification_report
data = pd.read_excel(r"C:\Users\ASUS\Downloads\skin dataset ml 7.xlsx")
target_column = data.columns[-1]
le = LabelEncoder()
for col in data.columns:
    if data[col].dtype == 'object':
        data[col] = le.fit_transform(data[col].astype(str))
X = data.drop(target_column, axis=1)
y = data[target_column]
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y
)
scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)
svm = SVC(kernel='rbf', C=10, gamma='scale', class_weight='balanced')
svm.fit(X_train, y_train)
y_pred = svm.predict(X_test)
print("Accuracy:", accuracy_score(y_test, y_pred))
print("\nConfusion Matrix:\n", confusion_matrix(y_test, y_pred))
print("\nClassification Report:\n", classification_report(y_test, y_pred, zero_division=0))
cm = confusion_matrix(y_test, y_pred)
plt.imshow(cm, cmap=plt.cm.Blues)
plt.title("Confusion Matrix")
```

```
plt.colorbar()

classes = np.unique(y)

tick_marks = np.arange(len(classes))

plt.xticks(tick_marks, classes)

plt.yticks(tick_marks, classes)

plt.xlabel("Predicted")

plt.ylabel("Actual")

for i in range(cm.shape[0]):

   for j in range(cm.shape[1]):

 plt.text(j, i, cm[i, j], ha="center", va="center", color="red")

plt.show()
```

OUTPUT:

```
Accuracy: 0.3

Confusion Matrix:
 [[0 0 1 1 0 0]
 [0 0 0 0 0 1]
 [1 0 1 0 0 0]
 [0 0 0 1 0 0]
 [1 0 0 0 0 0]
 [2 0 0 0 0 1]]

Classification Report:
              precision    recall  f1-score   support

           1       0.00      0.00      0.00         2
           2       0.00      0.00      0.00         1
           4       0.50      0.50      0.50         2
           5       0.50      1.00      0.67         1
           6       0.00      0.00      0.00         1
           7       0.50      0.33      0.40         3

    accuracy                           0.30        10
   macro avg       0.25      0.31      0.26        10
weighted avg       0.30      0.30      0.29        10
```



INPUT :8

```
import numpy as np
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler, LabelEncoder
from sklearn.metrics import accuracy_score
def sigmoid(x):
    return 1 / (1 + np.exp(-np.clip(x, -500, 500)))
def sigmoid_derivative(x):
    return x * (1 - x)
data = pd.read_csv(r"C:\Users\ASUS\Downloads\medical_insurance.csv")
print("Original Data:")
print(data.head())
label_encoders = {}
for column in data.columns:
    if data[column].dtype == 'object':
        le = LabelEncoder()
        data[column] = le.fit_transform(data[column])
```

```
    label_encoders[column] = le
print("\nEncoded Data:")
print(data.head())
X = data.drop(columns=['smoker']).values
y = data['smoker'].values.reshape(-1, 1)
X_train, X_test, y_train, y_test = train_test_split(
  X, y, test_size=0.2, random_state=42
)
scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)
input_neurons = X_train.shape[1]
hidden_neurons = 10
output_neurons = 1
np.random.seed(42)
W1 = np.random.randn(input_neurons, hidden_neurons)
b1 = np.zeros((1, hidden_neurons))
W2 = np.random.randn(hidden_neurons, output_neurons)
b2 = np.zeros((1, output_neurons))
learning_rate = 0.01
epochs = 1000
for epoch in range(epochs):
hidden_input = np.dot(X_train, W1) + b1
 hidden_output = sigmoid(hidden_input)
final_input = np.dot(hidden_output, W2) + b2
predicted_output = sigmoid(final_input)
error = y_train - predicted_output
d_output = error * sigmoid_derivative(predicted_output)
d_hidden = d_output.dot(W2.T) * sigmoid_derivative(hidden_output)
W2 += hidden_output.T.dot(d_output) * learning_rate
b2 += np.sum(d_output, axis=0, keepdims=True) * learning_rate
```

```
W1 += X_train.T.dot(d_hidden) * learning_rate

b1 += np.sum(d_hidden, axis=0, keepdims=True) * learning_rate

 if epoch % 100 == 0:

      loss = np.mean(error ** 2)

      print(f"Epoch {epoch}, Loss: {loss:.4f}")

hidden_test = sigmoid(np.dot(X_test, W1) + b1)

final_test = sigmoid(np.dot(hidden_test, W2) + b2)

y_pred = (final_test > 0.5).astype(int)

accuracy = accuracy_score(y_test, y_pred)

print("\nTest Accuracy:", accuracy)
```

OUTPUT:

```
Original Data:
   age     sex     bmi  children smoker     region      charges
0   19  female  27.900         0    yes  southwest  16884.92400
1   18    male  33.770         1     no  southeast   1725.55230
2   28    male  33.000         3     no  southeast   4449.46200
3   33    male  22.705         0     no  northwest  21984.47061
4   32    male  28.880         0     no  northwest   3866.85520

Encoded Data:
   age  sex     bmi  children  smoker  region      charges
0   19    0  27.900         0       1       3  16884.92400
1   18    1  33.770         1       0       2   1725.55230
2   28    1  33.000         3       0       2   4449.46200
3   33    1  22.705         0       0       1  21984.47061
4   32    1  28.880         0       0       1   3866.85520
Epoch 0, Loss: 0.2231
Epoch 100, Loss: 0.0290
Epoch 200, Loss: 0.0272
Epoch 300, Loss: 0.0260
Epoch 400, Loss: 0.0252
Epoch 500, Loss: 0.0246
Epoch 600, Loss: 0.0240
Epoch 700, Loss: 0.0234
Epoch 800, Loss: 0.0229
Epoch 900, Loss: 0.0223

Test Accuracy: 0.9585585585585585
```

INPUT :9

```
import pandas as pd

from mlxtend.preprocessing import TransactionEncoder

from mlxtend.frequent_patterns import apriori, association_rules

df = pd.read_csv(r"C:\Users\ASUS\Downloads\Assignment.csv")

transactions = df.groupby("BillNo")["itemName"].apply(list).tolist()
```

```
te = TransactionEncoder()
encoded = te.fit(transactions).transform(transactions)
df_encoded = pd.DataFrame(encoded, columns=te.columns_)
frequent_items = apriori(
    df_encoded,
    min_support=0.02,
    use_colnames=True
)
print("Frequent Itemsets:")
print(frequent_items)
rules = association_rules(
    frequent_items,
    metric="confidence",
    min_threshold=0.5
)
print("\nAssociation Rules:")
print(rules)
```

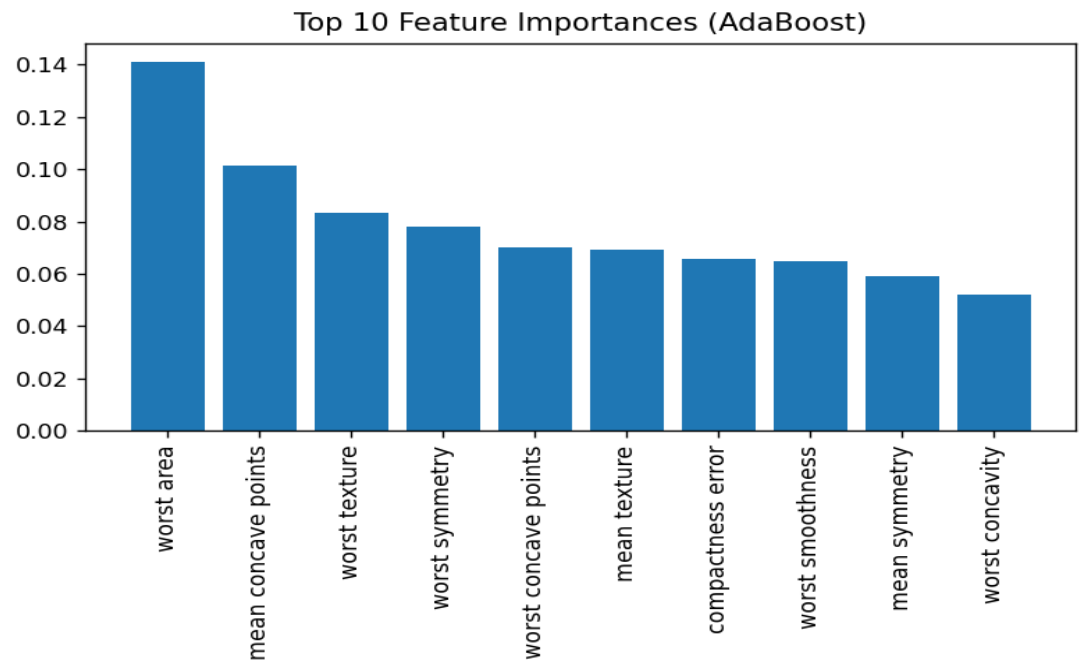
OUTPUT:

```
Frequent Itemsets:
    support                itemsets
0       0.8                 (Bread)
1       0.5                (Butter)
2       0.3                  (Eggs)
3       0.2                   (Jam)
4       0.7                  (Milk)
5       0.3         (Butter, Bread)
6       0.3           (Eggs, Bread)
7       0.2            (Bread, Jam)
8       0.5           (Bread, Milk)
9       0.1           (Butter, Jam)
10      0.3          (Butter, Milk)
11      0.3            (Eggs, Milk)
12      0.1    (Butter, Bread, Jam)
13      0.1   (Butter, Bread, Milk)
14      0.3     (Eggs, Bread, Milk)

Association Rules:
      antecedents      consequents  ...  certainty  kulczynski
0        (Butter)          (Bread)  ...  -1.000000    0.487500
1          (Eggs)          (Bread)  ...   1.000000    0.687500
2           (Jam)          (Bread)  ...   1.000000    0.625000
3         (Bread)           (Milk)  ...  -0.250000    0.669643
4          (Milk)          (Bread)  ...  -0.428571    0.669643
5           (Jam)         (Butter)  ...   0.000000    0.350000
6        (Butter)           (Milk)  ...  -0.333333    0.514286
7          (Eggs)           (Milk)  ...   1.000000    0.714286
8   (Butter, Jam)          (Bread)  ...   1.000000    0.562500
9    (Bread, Jam)         (Butter)  ...   0.000000    0.350000
10          (Jam)   (Butter, Bread)  ...   0.285714    0.416667
11  (Eggs, Bread)           (Milk)  ...   1.000000    0.714286
12   (Eggs, Milk)          (Bread)  ...   1.000000    0.687500
13  (Bread, Milk)           (Eggs)  ...   0.428571    0.800000
14         (Eggs)    (Bread, Milk)  ...   1.000000    0.800000

[15 rows x 14 columns]
```

INPUT :10

```
import numpy as np
import matplotlib.pyplot as plt
from sklearn.datasets import load_breast_cancer
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier
from sklearn.ensemble import AdaBoostClassifier
from sklearn.metrics import accuracy_score
data = load_breast_cancer()
X = data.data
y = data.target
feature_names = data.feature_names
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)
base_model = DecisionTreeClassifier(max_depth=1, random_state=42)
base_model.fit(X_train, y_train)
y_pred_base = base_model.predict(X_test)
base_accuracy = accuracy_score(y_test, y_pred_base)
boost_model = AdaBoostClassifier(
    estimator=DecisionTreeClassifier(max_depth=1),
    n_estimators=50,
    learning_rate=1.0,
    random_state=42
)
boost_model.fit(X_train, y_train)
y_pred_boost = boost_model.predict(X_test)
boost_accuracy = accuracy_score(y_test, y_pred_boost)
print("Base Learner Accuracy      :", base_accuracy)
print("AdaBoost Accuracy         :", boost_accuracy)
importances = boost_model.feature_importances_
```

```
indices = np.argsort(importances)[::-1][:10]
print("\nTop 10 Important Features:")
for i in indices:
    print(f"{feature_names[i]} : {importances[i]:.4f}")
plt.figure()
plt.bar(range(10), importances[indices])
plt.xticks(range(10), feature_names[indices], rotation=90)
plt.title("Top 10 Feature Importances (AdaBoost)")
plt.tight_layout()
plt.show()
```

OUTPUT:

```
Base Learner Accuracy       : 0.8947368421052632
AdaBoost Accuracy           : 0.9649122807017544

Top 10 Important Features:
worst area : 0.1412
mean concave points : 0.1015
worst texture : 0.0832
worst symmetry : 0.0781
worst concave points : 0.0702
mean texture : 0.0693
compactness error : 0.0658
worst smoothness : 0.0649
mean symmetry : 0.0592
worst concavity : 0.0521
```